Dynamic Value Model for ARC-AGI-3

Jason Feng
https://x.com/iamjasonfeng
August 13, 2026

Abstract

This paper presents Dynamic Value Model (DVM), a lightweight test-time method for interactive agents. During every action call, the acting model maintains a visible, persistent JSON value model that assigns and continually revises value judgments for candidate hypotheses, policies, subgoals, tests, actions, and possible states. It estimates value across immediate, short-term, medium-term, long-term, and ultra-long-term horizons while considering information value, potential upside and downside, risk, recoverability, evidence, uncertainty, and whether that value transfers to later levels. Before acting, the model explains how its current value judgments inform the next action. DVM complements rather than replaces the world model: the world model represents what the environment contains and how it behaves, while the value model represents what is worth pursuing and why. It requires no weight updates, auxiliary model calls, search branches, or environment copies. On the 25 public ARC-AGI-3 games, DeepSeek V4 Flash with DVM scored 14.38%, compared with 11.02% for Policy Superposition and 6.01% for the baseline. These single-run comparisons are preliminary; the earlier two runs also shared a temporary API-balance outage that did not affect DVM.

1. Introduction

ARC-AGI-3 evaluates agents in unfamiliar interactive environments whose rules and objectives are initially hidden.[1] An agent must learn from costly actions, preserve useful discoveries, revise mistaken interpretations, transfer knowledge across levels, and still act efficiently.

The Tufa Labs Duck harness already gives its action agent a persistent world model.[2] That memory helps the model represent entities, action effects, goals, uncertainty, and plans. A world model, however, does not necessarily tell the agent which uncertain direction is worth pursuing, why a probe may be useful, whether a risky action is recoverable, or how the value of a candidate changes across different time horizons.

This distinction motivated DVM. The method maintains a separate canonical value model throughout a game. After observing action outcomes, the same model that plays the game revises its estimates and states how the current value model informs its next action. The representation is natural-language JSON rather than a learned neural value function.

The framing was partly inspired by Ilya Sutskever's discussion of value functions with Dwarkesh Patel. Sutskever described value functions as a way to judge whether an intermediate direction is going well or badly before waiting for a final outcome.[7] DVM explores a deliberately lightweight test-time analogue of that intuition for language-model agents. It is not reinforcement learning, does not estimate expected return in the formal sense, and does not claim calibrated values.

In one public evaluation, DeepSeek V4 Flash with DVM scored 14.38%. I report that run as an observed system result, not as proof that DVM caused the score. The main contributions of this report are the method, its implementation, its instrumentation, and qualitative evidence showing how the value model evolved in a successful trajectory.

2. Dynamic Value Model

2.1 World Model and Value Model

DVM explicitly separates two kinds of memory:

- The **world model** describes what exists, how actions affect the environment, what the goal may be, and what remains uncertain.
- The **value model** appraises candidate policies, hypotheses, subgoals, tests, actions, and possible resulting states according to their usefulness for progress and action efficiency.

For each decision-relevant candidate, the prompt asks the model to consider:

- immediate value for the next action and outcome;
- short-term value for the next several actions or current subgoal;
- medium-term value for completing the current level;
- long-term value for later levels;
- ultra-long-term value for understanding and completing the whole game;
- information value, upside, downside, recoverability, evidence, uncertainty, and transferability.

Numerical scores may range from `-5` to `+5`, while `unknown` is permitted when the evidence does not justify precision. Risk is descriptive rather than a veto: the prompt permits recoverable risk when its information value or potential upside warrants exploration.

This use of the term **value model** is intentionally operational. In classical reinforcement learning, a value function estimates expected future return under a policy.[6] DVM instead asks an LLM to maintain explicit, revisable judgments in language. Its values are heuristic statements and scores, not trained or statistically calibrated predictions.

2.2 Always-Active Revision

One canonical DVM begins on the first action call and remains active for the entire game. After each observed outcome, the model may preserve, revise, split, merge, add, or remove entries. It is instructed to maintain one current model rather than an append-only collection of conflicting versions.

At a level transition, the complete DVM is carried forward. The model reassesses which lessons transfer, retires obsolete level-specific entries, preserves unresolved questions, and initializes estimates for the new level. Transfer status may be `likely level-specific`, `likely transferable`, `mixed or conditional`, or `inconclusive`.

Before each Python tool call, the model emits a visible JSON envelope:

```
DVM update JSON:
{
  "value_model": { ... },
  "value_guided_decision": "..."
}
```

The `value_model` is required and contains the complete current DVM. The optional `value_guided_decision` states how the model informs or is being adapted for the current action. The outer schema is strict, but the internal structure remains model-controlled.

2.3 Lightweight Integration

I implemented DVM as a notebook-level extension to the shareable Tufa Duck harness. DVM does not modify the attached Tufa source bundle, system prompt, Python tool, analyzer timeout, concurrency, benchmark loop, or scorecard lifecycle. It adds instructions to ordinary per-turn user prompts and stores each valid visible revision in isolated per-game state.

The instrumentation is fail-open. If a response omits or malforms its DVM block, gameplay continues and the last valid model remains available. Hidden reasoning is never imported into persistent memory.

DVM adds no auxiliary model calls. The same DeepSeek response updates the world model, updates DVM, and invokes the existing Python tool. This distinguishes the method from systems that use a separate critic, evaluator, simulator, or search worker.

3. Qualitative Example

The following abridged outputs come from game `ar25-0c556536`, which the system completed in 317 actions across all eight levels. The transcript contains 70 DeepSeek calls, 70 valid visible DVM blocks, and no DVM contract violation.

3.1 Appraising the First Probe

On the initial frame, the model preferred free structural inspection over an immediate environment action:

```
{
  "candidates": {
    "inspect_board": {
      "immediate": 4,
      "short": 3,
      "medium": 4,
      "info_value": 5,
      "risk": "none"
    },
    "probe_arrow_or_space": {
      "immediate": 1,
      "short": 3,
      "medium": 3,
      "info_value": 4,
      "risk": "low, recoverable"
    }
  }
}
```

It then stated:

> **Value-guided decision:** Highest-value first action is structural inspection (free, zero-risk) to seed the world model; probing actions only after that.

This is a simple example, but it shows the intended separation. The world model described the visible structures; DVM compared the prospective usefulness of two ways to proceed.

3.2 Revising After Evidence

After a `RIGHT` action moved two objects in opposite horizontal directions, the model reduced confidence in its first interpretation and preferred exact inspection over another blind probe:

```
{
  "hypotheses": {
    "mirror_game": {
      "idea": "S is a mirror axis; player B and mirrored c move symmetrically",
      "confidence": 0.35
    },
    "wall_slide": {
      "idea": "objects slide until adjacent to S wall",
      "confidence": 0.2
    }
  },
  "candidates": {
    "exact_diff_analysis": {
      "immediate": 4,
      "info_value": 5,
      "risk": "none"
    },
    "probe_DOWN_or_LEFT": {
      "immediate": 2,
      "info_value": 4,
      "risk": "low, recoverable"
    }
  }
}
```

> **Value-guided decision:** Before any further probe, run a free exact-diff analysis; the earlier hand-count of the ASCII is unreliable and the mirror hypothesis needs precise per-color displacement.

The subsequent analysis confirmed the mirror relationship. DVM then shifted from information gathering to execution and assigned high medium-term value to a path expected to complete Level 1.

3.3 Carrying Value Across Levels

When Level 1 cleared, the next DVM retained the confirmed lessons while reopening uncertainty about their applicability:

```
{
  "level1_lessons": {
    "mirror_rule": {
      "status": "confirmed in L1",
      "transfer": "likely transferable"
    },
    "goal_rule": {
      "status": "level completes when mirror block exactly covers Y shape",
      "transfer": "likely transferable"
    },
    "col63_counter": {
      "status": "HUD move counter",
      "transfer": "likely transferable"
    }
  },
  "level2_hypotheses": {
    "mirror_of_L1": {
      "confidence": 0.5
    },
    "two_pairs": {
      "confidence": 0.5,
      "needs": "exact bboxes from segmentation"
    }
  }
}
```

The model later revised the mechanic several times, learned that `SPACE` changed which object was controlled, generalized from vertical to horizontal mirror axes, and used those lessons through Level 8. Its final visible decision was:

> **Value-guided decision:** Finish B1's translation: RIGHT x11.

That action batch completed the game. This trajectory does not isolate DVM's causal contribution, but it demonstrates that the mechanism was present, dynamic, cross-level, and connected to real actions.

4. Evaluation

4.1 Setup

I evaluated the system once on all 25 public ARC-AGI-3 games in online mode. The acting model was DeepSeek V4 Flash with thinking enabled and maximum reasoning effort.[3] The maximum output setting was 384,000 tokens, the context window was one million tokens, and gameplay concurrency was 28.

DeepSeek was text-only in this configuration, so each frame was converted deterministically into an exact letter-coded grid with dimensions and a color legend. Tufa's ordinary Python inspection and environment-action tools remained available.

The implementation and runnable notebook are public:

- DeepSeek V4 Flash DVM on Kaggle
- ARC-AGI-3 scorecard

4.2 Descriptive Comparison

The baseline and Policy Superposition results come from separate runs using the same model family, Tufa harness, and 25-game public evaluation.[4][5] They are useful reference points, but they are not repeated estimates of expected performance.

Metric	Baseline	Policy Superposition	DVM
Public games	25	25	25
Mean score	6.01%	11.02%	14.38%
Median score	3.57%	4.76%	4.76%
Completed levels	42	52	58
Games completing at least one level	21	24	21
Fully solved games	0	0	1
Total environment actions	6,523	6,484	6,806
Total reported generated tokens	7,706,572	8,409,852	9,326,981
Duration	2h 17m 19s	2h 16m 18s	2h 15m 5s
Temporary API-balance outage	Yes	Yes	No

DVM's observed mean was 8.37 percentage points above the baseline and 3.36 points above Policy Superposition. It completed 16 more levels than the baseline and six more than Policy Superposition. The distribution matters, however: DVM progressed in fewer games than Policy Superposition, while its higher aggregate score came from deeper progress on several games, including one complete solution.

The baseline and Policy Superposition runs experienced the same approximately 10-minute-45-second API-balance outage, with four exhausted analyzer calls in every game and successful calls after recovery in all 25 games. DVM did not experience that outage. This makes the three-way comparison informative but not controlled: the clean DVM run had an infrastructure advantage that cannot be converted into a reliable score correction, and stochastic trajectory differences remain a major confound.

The two deepest trajectories were `ar25-0c556536`, which completed 8/8 levels and scored 100.00%, and `tu93-0768757b`, which completed 8/9 levels and scored 61.92%. The aggregate is therefore influenced by large successes. Excluding `ar25` lowers the mean to 10.81%; excluding both `ar25` and `tu93` lowers it to 8.59%.

The run itself was operationally clean. No game crashed, no analyzer request exhausted its retry budget, and there was no insufficient-balance incident. Four transient network exceptions were retried successfully.

4.3 Mechanism Compliance

The DVM event log contains 4,206 structured records. It records:

- 1,952 accepted visible value-model revisions;
- 1,872 explicit value-guided decisions;
- 243 prompt turns without a valid DVM block;
- 82 observed level states and 57 observed cross-level transitions.

Every observed transition carried the prior DVM forward. Sixteen of the 25 games had no DVM contract violations, and 21 had at most two. Four games accounted for 237 of the 243 violations, indicating concentrated rather than uniform noncompliance.

Across the complete transport log, 2,120 of 2,324 DeepSeek calls satisfied the visible Tufa-memory output contract. Some hidden reasoning contained memory-like labels, but hidden reasoning was never used as persistent memory. The accepted DVM revisions had a median length of 853 characters and a maximum of 4,296 characters.

The clean `ar25` transcript provides stronger qualitative evidence than format counts alone. All 70 calls produced valid DVM JSON, all eight levels were completed, and successive responses visibly revised both the model and the decision attributed to it.

5. Related Work

Classical value functions estimate expected future return and provide a basis for comparing policies or actions.[6] DVM borrows the idea that intermediate states and directions can be appraised before the final outcome, including the motivation expressed by Sutskever.[7] It does not learn a value function, perform temporal-difference updates, or claim probabilistic calibration.

Several language-agent systems use value judgments inside explicit search. Tree of Thoughts explores and evaluates multiple reasoning branches.[8] Reasoning via Planning uses an LLM as both a reasoning agent and world model within Monte Carlo tree search guided by task-specific rewards.[9] Language Agent Tree Search combines Monte Carlo tree search, LM-powered value functions, and self-reflection.[10] DVM differs structurally: it has no tree, rollback, branch execution, separate evaluator, or additional inference call. The actor maintains one revisable value model while following one real trajectory.

Reflexion stores verbal feedback in episodic memory so later trials can improve without weight updates.[11] DVM is also linguistic and test-time, but it is revised during the active trajectory and is meant to guide the next action rather than primarily summarize a completed attempt.

Internalizing the Future trains agents to verbalize prospective state rollouts and plan-conditioned success estimates described as textual analogues of Q-values.[12] That work is close in its explicit textual value framing, but it develops the capability through mid-training, supervised fine-tuning, and reinforcement learning. DVM is a prompt-and-memory intervention applied to an existing model at inference time.

WorldEvolver updates deployment-time memory from prediction-observation mismatches and uses selective foresight to improve planning.[13] Its primary object is a predictive world model. DVM instead places a separate appraisal layer alongside Tufa's existing world model. The distinction is central: prediction asks what an action will do; valuation asks why a candidate action, test, subgoal, or state may be worth pursuing.

Within ARC-AGI-3, Tycho constructs, verifies, repairs, and plans with executable world models.[14] Tycho also emphasizes metareasoning about when model construction or use is worth its cost. DVM is substantially lighter and less reliable: it uses a non-executable natural-language value model inside the ordinary actor, with no verifier or delegated builder. Tufa Duck supplies the persistent action loop on which DVM is layered.[2] The broader interaction pattern is related to ReAct's interleaving of reasoning, action, and observation.[15]

6. Interpretation

The run shows that a language model can maintain a distinct value-oriented representation during long interactive play. In the clean transcript, DVM did more than repeat the world model. It compared inspection with action, assigned information value to discriminating tests, described recoverability, revised transfer claims at level boundaries, and connected the resulting appraisal to the next tool call.

The strongest practical feature is low structural overhead. DVM adds visible context but no extra model requests, environment copies, search tree, learned critic, or weight updates. It can therefore be attached to an existing persistent agent loop without replacing that loop.

The run also reveals that the current implementation is only a rough language analogue of a value function. Scores were strongly concentrated at `+4` and `+5`. In the `ar25` transcript, numerical immediate and information values were common, while numerical long-term and ultra-long-term values were rare. Those horizons were usually represented as prose goals instead. Explicit evidence and confidence fields were also inconsistent, and later updates sometimes became compact plan trackers that partially duplicated the world model.

These weaknesses suggest that DVM may help through several overlapping effects: explicit valuation, better-organized memory, increased attention to information value, forced reconsideration, or simply more deliberate planning. The present run cannot separate them.

7. Limitations

1. The evaluation contains one stochastic run and does not estimate run-to-run variance.

2. There is no clean matched no-DVM control run from the same evaluation window.
3. Two deep trajectories substantially influence the mean score.
4. Format compliance does not prove that the model's scores were calibrated or causally useful.
5. DVM combines several components: multi-horizon scoring, information value, risk, transfer status, persistent revision, and explicit decision attribution. No ablation isolates them.
6. The method increases visible output and retained context even though it adds no model calls.
7. The model sometimes ignored the DVM contract, and a few games accounted for most violations.
8. The study covers only the 25 public ARC-AGI-3 games.

Repeated matched runs, component ablations, and qualitative audits across more game transcripts are needed before making a general performance claim.

8. Reproducibility and Artifacts

The Kaggle notebook preserves the complete implementation. Its generated artifacts include:

- `dvm_events.jsonl`, the structured mechanism event stream;
- `dvm_games/*/dvm_state.json`, isolated canonical state for each game;
- `dvm_summary.json`, aggregate and per-game statistics;
- `dvm_report.html`, a readable report with DVM excerpts;
- ordinary Tufa diagnostics and per-game transcripts.

These files make it possible to distinguish mechanism execution from benchmark outcome and to inspect whether the model actually produced and used visible DVM updates.

9. Acknowledgments

I developed DVM in collaboration with OpenAI Codex.[16]

10. Conclusion

DVM asks an interactive language-model agent to maintain a separate, always-active account of what may be valuable and why. It preserves that account across actions and levels, revises it from real outcomes, and requires the model to connect the current appraisal to its next action.

In one DeepSeek V4 Flash run over the 25 public ARC-AGI-3 games, the system scored 14.38%, completed 58 levels, and fully solved one game. The result is preliminary, but the instrumentation and successful transcript establish that the method operated in a substantive way. A persistent natural-language value model may be a useful lightweight complement to the world models already used by interactive language agents.

References

[1] ARC Prize Foundation. (2026). ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence.
[2] Tufa Labs. (2026). The Duck: ARC-AGI-3 inference harness. See also the shareable Kaggle notebook.
[3] DeepSeek. (2026). DeepSeek V4 Preview Release, Thinking Mode, and Models and Pricing.
[4] Feng, J. (2026). DeepSeek ARC-AGI-3 Online Baseline. Kaggle notebook. Scorecard.
[5] Feng, J. (2026). DeepSeek Superposition ARC-AGI-3 Online. Kaggle notebook. Scorecard.
[6] Sutton, R. S., and Barto, A. G. (2018). Reinforcement Learning: An Introduction, Second Edition. MIT Press.
[7] Sutskever, I., and Patel, D. (2025). Ilya Sutskever: We're Moving from the Age of Scaling to the Age of Research. Interview transcript; see the discussion of emotions and value functions beginning at 00:09:39.
[8] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models.
[9] Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., and Hu, Z. (2023). Reasoning with Language Model is Planning with World Model.
[10] Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. (2023). Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models.
[11] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning.
[12] Zhang, X., Zhou, Z., Qiao, L., Qin, Y., Li, K., Sun, X., Tan, X., Qu, C., and Qi, Y. (2026). Internalizing the Future: A Unified Agentic Training Paradigm for World Model Planning.
[13] Zhang, X., Zhang, W., Ng, S.-K., and Deng, Y. (2026). Self-Evolving World Models for LLM Agent Planning.
[14] Lehmann, J., Aioanei, A., and Vahdati, S. (2026). Tycho: Active Abstraction with Programmatic World Models for ARC-AGI-3.
[15] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models.
[16] OpenAI. Codex.